# HOUSEHOLD DEPENDENCY ANALYTICS & PREDICTION SYSTEM

## Academic Full Stack Development Project Report

# PRELIMINARY PAGES

## 1. TITLE PAGE

**Project Title:** HOUSEHOLD DEPENDENCY ANALYTICS & PREDICTION SYSTEM

**Document Type:** Full Stack Development Project Report

**Degree / Course:** Bachelor of Engineering in Computer Science & Engineering (Full Stack Development)

**Academic Year:** 2025 – 2026

**Student Name:** [Student Name Placeholder]

**USN / Roll Number:** [USN / Roll Number Placeholder]

**Department:** Department of Computer Science & Engineering

**Institution:** [Institution / College Name Placeholder]

**Project Guide / Mentor:** [Faculty Guide Name Placeholder]

**Date of Submission:** October 10, 2026

## 2. CERTIFICATE

This is to certify that the project report titled **"HOUSEHOLD DEPENDENCY ANALYTICS & PREDICTION SYSTEM"** is a bona fide record of the work carried out by **[Student Name Placeholder]** (USN: **[USN Placeholder]**) in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering in Computer Science & Engineering (Full Stack Development)** during the academic year 2025–2026.

_____

**Project Guide / Mentor**

[Faculty Name & Designation]

_____

**Head of Department (HOD)**

Department of Computer Science & Engineering

_____

**External Evaluator / Examiner**

## 3. DECLARATION

I hereby declare that the project report titled **"HOUSEHOLD DEPENDENCY ANALYTICS & PREDICTION SYSTEM"** submitted to the Department of Computer Science & Engineering is an authentic record of my original work under the supervision of **[Faculty Guide Name Placeholder]**.

I declare that this work has not formed the basis for the award of any degree, diploma, associate-ship, fellowship, or similar title at any other university or institution.

**Place:** [City Placeholder]

**Date:** October 10, 2026

**Signature of Student:** _____

## 4. ACKNOWLEDGEMENT

I express my deepest gratitude to my project guide, **[Faculty Guide Name Placeholder]**, for invaluable guidance, encouragement, continuous support, and insightful technical suggestions throughout the design, implementation, and testing of this project.

I extend my sincere thanks to the Head of Department, **[HOD Name Placeholder]**, and all faculty members of the Department of Computer Science & Engineering for providing state-of-the-art laboratory facilities and academic resources.

Finally, I express my appreciation to my peers and family members for their constant motivation and feedback during this project journey.

## 5. ABSTRACT

Modern households frequently face complex financial challenges due to shifting demographic dynamics, rising living expenses, unmonitored discretionary spending, and inadequate visibility into financial risk. Conventional personal finance software focuses primarily on basic transaction logging, lacking specialized mathematical tools to compute demographic dependency ratios, assess essential versus non-essential spending structures, or provide early warning indicator signals for household vulnerability.

To address these limitations, this project presents the **Household Dependency Analytics & Prediction System**, a production-grade 3-tier full-stack application built using a modern decoupled architecture. The system combines:

1. **Deterministic Analytics Engine:** Built into the Node.js/Express backend to compute exact financial and demographic metrics (e.g., Household Dependency Ratio, Savings Rate, Essential Expenditure Ratio, and Financial Condition classifications) under strict mathematical bounds.

2. **Supervised Machine Learning Microservice:** Developed in Python with FastAPI, serving a Random Forest classifier trained on synthetic household profiles (1,000 samples, 80/20 split, 95.0% accuracy) to predict household risk levels (**LOW**, **MEDIUM**, **HIGH**) and isolate contributing risk factors.

3. **Conversational AI Assistant (Google Gemini 2.5 Flash):** Integrated as a floating ChatGPT-style chatbot widget at the bottom-right corner of the web application, offering interactive, context-aware financial explanations while maintaining server-side API key protection.

4. **Read-Only Spring Boot Service:** Built with Java 17 and Spring Data JPA to provide scalable, read-only financial summaries directly from the relational MySQL database.

The application features a responsive React 18 user interface styled with a custom Biscoff Cream theme, interactive Chart.js visualizations, PDF report generation, and session persistence. Rigorous testing—comprising **109 Node.js Jest tests**, **21 Python Pytest tests**, and **9 Frontend Vitest tests** (total **139 passing tests**)—verifies the system's reliability, security, and correctness. This report provides a complete documentation of the project lifecycle from initial problem definition through system design, implementation, testing, results, deployment, and future scope.

## 6. TABLE OF CONTENTS


• **Preliminary Pages**

- 1. Title Page
- 2. Certificate
- 3. Declaration
- 4. Acknowledgement
- 5. Abstract
- 6. Table of Contents
- 7. List of Figures, Tables, and Abbreviations

• **Chapter 1: Introduction**

- 1.1 Background
- 1.2 Problem Statement
- 1.3 Motivation
- 1.4 Objectives
- 1.5 Scope
- 1.6 Existing System
- 1.7 Limitations of Existing Approach
- 1.8 Proposed System
- 1.9 Benefits
- 1.10 Report Organization

• **Chapter 2: Literature Review and Related Work**

• **Chapter 3: Requirement Analysis**

- 3.1 Stakeholders and Intended Users
- 3.2 Functional Requirements
- 3.3 Non-Functional Requirements
- 3.4 Hardware Requirements
- 3.5 Software Requirements
- 3.6 Use Cases
- 3.7 Feasibility Study
- 3.8 Constraints and Assumptions

• **Chapter 4: System Architecture and Design**

- 4.1 Overall Architecture
- 4.2 Architecture Explanation
- 4.3 Module Design

- 4.4 Data Flow Diagrams (DFD)
- 4.5 UML Diagrams
- 4.6 Authentication and Authorization Design
- 4.7 API Design Specification
- 4.8 Error Handling Architecture

• **Chapter 5: Database Design**

- 5.1 Database Overview
- 5.2 Entity Identification
- 5.3 Table Specifications
- 5.4 Entity-Relationship (ER) Diagram
- 5.5 Relationships and Normalization
- 5.6 Database Operations and Security
- 5.7 Sample Synthetic Data

• **Chapter 6: Technology Stack and Justification**
• **Chapter 7: Detailed Implementation**

- 7.1 Project Structure
- 7.2 Frontend Implementation
- 7.3 Backend Implementation
- 7.4 Authentication Implementation
- 7.5 Household Management Implementation
- 7.6 Expense Tracking Implementation
- 7.7 Deterministic Analytics Engine
- 7.8 Machine Learning Microservice
- 7.9 Risk Classification
- 7.10 Google Gemini AI Integration (Floating Chatbot Widget)
- 7.11 Report Generation
- 7.12 Frontend–Backend Communication Lifecycle
- 7.13 Spring Boot Microservice Implementation

• **Chapter 8: Analytics and Prediction Workflow**
• **Chapter 9: Security, Privacy, and Reliability**
• **Chapter 10: Testing and Validation**
• **Chapter 11: Results and Discussion**
• **Chapter 12: Deployment and Execution Guide**
• **Chapter 13: Limitations and Future Enhancements**

- **Chapter 14: Conclusion**
- **References and Appendices**

## 7. LIST OF FIGURES, TABLES, AND ABBREVIATIONS

##### List of Figures

1. Figure 4.1: Overall 3-Tier Microservice System Architecture
2. Figure 4.2: Level 0 Data Flow Diagram (DFD)
3. Figure 4.3: Level 1 Data Flow Diagram (DFD)
4. Figure 4.4: System Use Case Diagram
5. Figure 4.5: End-to-End Prediction and AI Chat Sequence Diagram
6. Figure 5.1: Database Entity-Relationship (ER) Diagram
7. Figure 7.1: Floating Gemini AI Chatbot Architecture
8. Figure 8.1: Complete End-to-End Processing Pipeline Diagram
9. Figure 11.1: System Dashboard Interface
10. Figure 11.2: Expenses Manager Interface
11. Figure 11.3: Analytics Engine Interface
12. Figure 11.4: ML Risk Prediction Interface
13. Figure 11.5: Floating Gemini Chatbot Widget Interface

##### List of Tables

1. Table 3.1: Hardware Requirements Specification
2. Table 3.2: Software Requirements Specification
3. Table 4.1: Comprehensive REST API Specification Table
4. Table 5.1: Database Table Summary
5. Table 5.2 to 5.8: MySQL Database Column Specifications
6. Table 6.1: Comprehensive Technology Stack Justification Table
7. Table 7.1: Deterministic Analytics Formulas & Definitions
8. Table 7.2: Random Forest Model Feature Specification Table
9. Table 10.1: Summary of Automated Test Suite Results (139 Total Tests)
10. Table 10.2: Executed Functional Test Matrix
11. Table 11.1: Objective-to-Result Mapping Matrix

##### Abbreviations

- **API:** Application Programming Interface
- **CRUD:** Create, Read, Update, Delete
- **DFD:** Data Flow Diagram
- **ER:** Entity-Relationship
- **FSD:** Full Stack Development
- **JPA:** Java Persistence API

- **JWT:** JSON Web Token
- **ML:** Machine Learning
- **REST:** Representational State Transfer
- **SQL:** Structured Query Language
- **UI/UX:** User Interface / User Experience

# CHAPTER 1: INTRODUCTION

## 1.1 Background

In contemporary socio-economic environments, effective household financial management is crucial for long-term stability and resilience. A household's financial vulnerability is shaped by the interplay between its demographic structure (number of earning members versus dependent children and elderly members) and its spending behavior (essential expenditure on food, housing, healthcare, and education versus non-essential discretionary spending).

Traditional financial advice often relies on general rules of thumb that fail to account for specific household earner-to-dependent dynamics. For example, two households with identical monthly incomes of INR 100,000 may exhibit drastically different financial risk profiles if Household A has 3 earning adults supporting 1 dependent, while Household B has 1 sole earner supporting 5 dependents (including 2 infants and 2 elderly members requiring medical care).

Understanding financial dependency ratios, essential spending commitments, and net savings rates empowers families to identify financial stress early, build adequate emergency funds, and optimize their budgets before unexpected economic shocks occur.

## 1.2 Problem Statement

Despite the proliferation of digital budgeting apps, households face significant challenges in managing financial risk due to:

1. **Fragmented Record Keeping:** Financial data is scattered across manual registers, spreadsheets, and banking apps without unified aggregation.

2. **Lack of Demographic Analysis:** Traditional tools track expense totals but ignore household member composition, failing to measure demographic dependency pressure.

3. **Manual Calculations & Calculation Errors:** Computing savings rates, essential expense ratios, and earner-to-dependent balances manually is tedious and error-prone.

4. **Absence of Risk Prediction:** Standard applications report past spending but provide no forward-looking Machine Learning risk assessment to classify household vulnerability.

5. **Generative AI Hallucination Risks:** Generic AI chatbots often invent ungrounded financial figures when asked for advice without strict underlying data boundaries.

The **Household Dependency Analytics & Prediction System** addresses these problems by providing a unified, secure web platform that pairs deterministic analytics with supervised Machine Learning and context-aware conversational AI.

## 1.3 Motivation

The primary motivation for this project is to develop a production-ready, full-stack software solution that bridges software engineering, database design, machine learning, and artificial intelligence. By combining a Node.js/Express core, a Python FastAPI ML microservice, a Spring Boot read-only summary service, a MySQL database, and Google Gemini AI, this project demonstrates how multi-tier microservice architectures solve real-world financial domain problems efficiently.

## 1.4 Objectives

The core objectives of the system are:

- **Centralized Household Management:** Provide secure multi-tenant CRUD capabilities for household profiles and demographics.
- **Categorized Expense Tracking:** Enable granular logging of essential and non-essential expenses across 11 predefined spending categories.
- **Deterministic Analytics Engine:** Implement mathematically rigorous backend calculations for Savings Rate, Dependency Ratio, Essential Ratio, and Financial Condition classifications.
- **Supervised ML Risk Prediction:** Deploy a Random Forest prediction service to classify financial vulnerability (LOW, MEDIUM, HIGH) with high confidence.
- **Interactive Floating AI Chatbot:** Mount a floating Gemini AI chatbot at the layout level for real-time financial Q&A grounded in the active household's financial metrics.
- **PDF Report Generation:** Compile comprehensive vector PDF financial reports summarizing household status, ML metrics, and strategic recommendations.
- **High Test Quality:** Validate all business logic, endpoints, and UI components with comprehensive automated test suites.

## 1.5 Scope

The system supports:

- User registration, authentication, JWT session management, and household profile isolation.
- Detailed demographic logging (total members, earners, dependents, children, elderly).
- Categorized expense logging with essential vs non-essential indicators.
- Instant analytics calculation and interactive Chart.js visualization.
- Supervised ML risk inference via Python FastAPI microservice.
- Interactive conversational AI assistance via a floating widget.
- PDF report downloads and monthly financial snapshot tracking.

**Boundaries:** The system is an educational assessment and decision support tool; it does not perform direct bank account synchronization, payment gateway processing, or certified legal investment advice.

## 1.6 Existing System

Conventional household budgeting app workflows typically involve:

1. Manual ledger entry or basic mobile spreadsheets.
2. Manual sum aggregation without categorization standards.
3. Generic bar charts showing total monthly expenses versus income.

## 1.7 Limitations of Existing Approach

- **No Dependency Analysis:** Ignore earner-to-dependent ratios.

- **No Predictive Risk Scoring:** Backward-looking only; no predictive modeling.
- **Static Advice:** Lacks interactive, personalized AI guidance tailored to demographic realities.
- **Data Inconsistencies:** Spreadsheets lack database constraints and normalization.

## 1.8 Proposed System

The proposed system addresses these gaps using a modern 3-tier architecture:

- **React 18 Frontend:** Polished Biscoff cream theme with interactive visual analytics and layout-level floating chatbot.
- **Node.js/Express Backend:** Handles auth, CRUD operations, deterministic calculations, and report generation.
- **MySQL Database:** Relational schema enforcing referential integrity and check constraints.
- **Python FastAPI ML Microservice:** Serves a Random Forest risk model.
- **Spring Boot 3.2.5 Service:** Scalable read-only summary endpoint.
- **Google Gemini 2.5 Flash:** Provides grounded financial Q&A via a server-protected API integration.

## 1.9 Benefits

- Unified visibility into household financial health and demographic vulnerability.
- Automated zero-error mathematical analytics.
- Objective risk classification via supervised machine learning.
- Contextual advice via an elegant floating chatbot without exposing secret API keys.
- Production-grade reliability backed by 139 passing automated tests.

## 1.10 Report Organization

This report is organized into 14 chapters:

- **Chapter 1:** Introduction, problem statement, motivation, and scope.
- **Chapter 2:** Literature review and technical background.
- **Chapter 3:** Requirement analysis, user roles, functional/non-functional specs.
- **Chapter 4:** System architecture, module breakdown, DFDs, UML diagrams, REST APIs.
- **Chapter 5:** Database design, ER diagram, SQL schema, normalization.
- **Chapter 6:** Technology stack analysis and justification matrix.
- **Chapter 7:** Detailed implementation across frontend, backend, ML, Gemini chatbot, and Spring Boot.
- **Chapter 8:** End-to-end analytics and prediction workflow with worked numerical example.
- **Chapter 9:** Security, privacy, rate limiting, and data isolation.
- **Chapter 10:** Comprehensive testing strategy and evidence (139 passing tests).
- **Chapter 11:** Results, UI screen demonstrations, objective mapping.
- **Chapter 12:** Step-by-step Windows setup, installation, and deployment guide.
- **Chapter 13:** System limitations, future scope, and ethical considerations.

• **Chapter 14:** Conclusion and final evaluation summary.

# CHAPTER 2: LITERATURE REVIEW AND RELATED WORK

## 2.1 Household Budgeting and Demographic Financial Risk

Financial economics literature emphasizes that financial vulnerability is not determined solely by income level, but by the ratio of fixed essential liabilities to total liquid income (Mullainathan & Shafir, 2013). Studies by the Consumer Financial Protection Bureau (CFPB, 2017) indicate that households maintaining a net savings rate above 20% and an essential expenditure ratio below 60% demonstrate significantly higher resilience against sudden unemployment or medical emergencies.

## 2.2 Dependency Ratios in Microeconomics

Demographic dependency ratios are traditionally applied at macroeconomic scales to measure the ratio of non-working age populations (under 15 and over 65) to the working-age population (World Bank, 2022). Applying dependency ratio formulas at the individual household level—specifically measuring dependent members per earning member—provides a direct metric of demographic financial pressure (Modigliani's Life-Cycle Hypothesis).

## 2.3 Machine Learning in Personal Risk Assessment

Supervised classification algorithms, particularly Random Forests and Gradient Boosted Trees, have proven highly effective in credit scoring and financial risk modeling (Breiman, 2001; Hand & Henley, 1997). Random Forest ensembles combine multiple decision trees built on bootstrapped feature subsets, offering high accuracy, non-linear feature interaction capture, and resistance to overfitting.

## 2.4 Generative AI and Grounded Context in Financial Support

Recent advancements in Large Language Models (LLMs) such as Google Gemini 2.5 Flash enable natural language interaction for complex domains (Vaswani et al., 2017; Gemini Team, 2024). However, unconstrained LLMs are prone to hallucinating factual errors. Combining deterministic backend data retrieval (Retrieval-Augmented Context Injection) with LLM prompts ensures that AI responses remain strictly grounded in verified financial numbers.

## 2.5 Comparative Analysis of System Approaches

| Feature / System | Basic Budgeting Apps | Commercial Spreadsheets | Generic AI Chatbots | Proposed System |

| :--- | :--- | :--- | :--- | :--- |

| Centralized Database | Yes | No (Local files) | No | Yes (MySQL 8.0) |

| Categorized Expense Logging | Yes | Manual | No | Yes (11 Categories) |

| Demographic Dependency Ratios | No | Manual formulas | No | Yes (Automated) |

| Supervised ML Risk Prediction | No | No | No | Yes (Random Forest ML) |

| Floating Contextual AI Chatbot | No | No | Un-grounded | Yes (Gemini 2.5 Flash) |

| Read-Only Microservice API | No | No | No | Yes (Spring Boot 3.2.5) |

# CHAPTER 3: REQUIREMENT ANALYSIS

## 3.1 Stakeholders and Intended Users

- **Primary Users (Household Members / Heads):** Manage household profiles, log expenses, review financial analytics, run ML risk predictions, ask Gemini chatbot questions, and download PDF reports.
- **System Administrators / Developers:** Monitor backend logs, maintain database schemas, update ML model artifacts, and manage environment configurations.

## 3.2 Functional Requirements

1. **User Authentication & Authorization (FR-1):** Users must register with name, email, and password (hashed via bcrypt), sign in securely, receive JWT session tokens, and access only their owned households.

2. **Household Profile Management (FR-2):** Users must create, view, update, and delete household profiles specifying total members, earning members, dependent members, children, elderly, and monthly income.

3. **Expense Management (FR-3):** Users must log expenses assigned to 11 categories with essential/non-essential flags, triggering instant recalculation of total expenses and savings.

4. **Deterministic Analytics Engine (FR-4):** System must compute exact mathematical metrics (Savings Rate, Dependency Ratio, Essential Ratio, Expense Ratio, Financial Condition, Dependency Level).

5. **Zero-Earner Edge Case Handling (FR-5):** When earning members = 0 or income = 0, system must output "Not Computable" or "NO_INCOME" warning badges without throwing runtime divide-by-zero errors.

6. **ML Risk Prediction (FR-6):** System must format 10 feature vectors, dispatch requests to the Python FastAPI microservice, and return risk classification (LOW, MEDIUM, HIGH), confidence scores, and top risk factors.

7. **Floating Gemini AI Chatbot (FR-7):** System must render a floating widget at the bottom-right corner across all dashboard pages, sending questions to backend /api/ai/chat with authorized household context and displaying replies with session storage persistence.

8. **PDF Report Generation (FR-8):** System must compile server-side vector PDF reports summarizing analytics, predictions, expense breakdowns, and recommendations.

9. **Spring Boot Summary Endpoint (FR-9):** System must provide a read-only Spring Boot service endpoint on port 8081 for fast financial summary queries.

## 3.3 Non-Functional Requirements

- **Security (NFR-1):** Passwords hashed with bcrypt (10 rounds). JWT secret keys stored in environment variables. Gemini API keys strictly hidden on backend.
- **Performance (NFR-2):** Analytics API response time < 50ms. ML prediction response time < 150ms. Page load time < 1 second.
- **Reliability (NFR-3):** 100% mathematical accuracy for analytics. Fallback responses for ML or AI service offline states.
- **Usability (NFR-4):** Responsive Biscoff cream visual design adapted for desktop and mobile breakpoints. Smooth animations and clear error state feedback.

- **Data Integrity (NFR-5):** Database foreign keys with ON DELETE CASCADE, check constraints on non-negative values and earner counts.

## 3.4 Hardware Requirements Specification

| Hardware Component | Minimum Requirement | Recommended Specification |

| :--- | :--- | :--- |

| Processor (CPU) | Dual-Core 2.0 GHz | Quad-Core 2.8 GHz or higher |

| System Memory (RAM) | 4 GB RAM | 8 GB or 16 GB DDR4 RAM |

| Storage (Disk) | 10 GB available SSD space | 25 GB NVMe SSD |

| Display Resolution | 1280 x 720 (HD) | 1920 x 1080 (Full HD) responsive |

| Network Interface | Standard Broadband / Wi-Fi | High-speed Broadband |

## 3.5 Software Requirements Specification

| Software Component | Technology / Tool Used | Version / Details |

| :--- | :--- | :--- |

| Operating System | Microsoft Windows 11 / Linux | 64-bit OS |

| Frontend Framework | React.js, Vite | React 18.3.1, Vite 5.2.11 |

| Backend Runtime | Node.js, Express.js | Node 18.x / 20.x, Express 4.19 |

| Database Engine | MySQL Server | MySQL 8.0+ |

| ML Microservice | Python, FastAPI, Uvicorn | Python 3.12, FastAPI 0.111 |

| ML Libraries | scikit-learn, pandas, joblib | scikit-learn 1.5.0, pandas 2.2 |

| Spring Boot Service | Java JDK, Spring Boot | Java 17, Spring Boot 3.2.5 |

| AI Integration | Google Gemini API | @google/genai (gemini-2.5-flash) |

| Testing Tools | Jest, Pytest, Vitest | Jest 29.7, Pytest 9.1, Vitest 4.1 |

# CHAPTER 4: SYSTEM ARCHITECTURE AND DESIGN

## 4.1 Overall Architecture

## 4.2 Module Design

1. **Authentication Module:** Handles user registration, password hashing (bcryptjs), JWT generation, and token authentication middleware.

2. **Household Management Module:** Performs CRUD operations on household demographic records, ensuring user ownership boundaries (user_id).

3. **Expense Management Module:** Enables adding, updating, and deleting expenses across 11 categories with essential/non-essential designations.

4. **Deterministic Analytics Engine:** Executes server-side mathematical evaluations for income, savings, expense ratios, dependency ratios, and financial status flags.

5. **Machine Learning Microservice Module:** Formats 10-feature vectors, calls Python FastAPI /predict, handles scaling, and returns risk classifications (LOW, MEDIUM, HIGH).

6. **Floating Gemini AI Chatbot Module:** Mounts layout-level chatbot widget, dispatches user prompts to /api/ai/chat, attaches household context, and persists chat sessions in sessionStorage.

7. **Report Generation Module:** Server-side PDF compilation of comprehensive financial reports.

8. **Spring Boot Summary Module:** Java 17 read-only REST microservice querying financial summaries on port 8081.

## 4.3 API Design Specification Table

| Method | Endpoint | Auth | Purpose | Success Response |

| :--- | :--- | :--- | :--- | :--- |

| POST | /api/auth/register | No | User signup | 201 Created, {token, user} |

| POST | /api/auth/login | No | User signin | 200 OK, {token, user} |

| GET | /api/auth/me | Yes | Get active user profile | 200 OK, {user} |

| GET | /api/households | Yes | List user households | 200 OK, [households] |

| POST | /api/households | Yes | Create household | 201 Created, {household} |

| GET | /api/households/:id | Yes | Get household details | 200 OK, {household} |

| PUT | /api/households/:id | Yes | Update household | 200 OK, {household} |

| DELETE | /api/households/:id | Yes | Delete household | 200 OK, {message} |

| GET | /api/households/:id/expenses | Yes | List expenses | 200 OK, [expenses] |

| POST | /api/households/:id/expenses | Yes | Add expense | 201 Created, {expense} |

| DELETE | /api/households/:id/expenses/:expId | Yes | Delete expense | 200 OK, {message} |

| GET | /api/households/:id/analytics | Yes | Get analytics metrics | 200 OK, {analytics} |

| GET | /api/households/:id/prediction | Yes | Get ML risk prediction | 200 OK, {prediction} |

| POST | /api/ai/chat | Yes | Interactive Gemini Chat | 200 OK, {reply, disclaimer} |

| GET | /api/spring/households/:id/summary | Optional | Spring Boot summary | 200 OK, {summary} |

# CHAPTER 5: DATABASE DESIGN

## 5.1 Database Overview

- **Database Engine:** MySQL Server 8.0+
- **Database Name:** `household_dependency`
- **Character Set / Collation:** `utf8mb4` / `utf8mb4_unicode_ci`
- **Storage Engine:** InnoDB

## 5.2 Table Specifications

### 1. `users` Table

| Column Name | Data Type | Key | Nullable | Default | Description |
| --- | --- | --- | --- | --- | --- |
| `id` | BIGINT UNSIGNED | PK | No | AUTO_INCREMENT | Unique user identifier |
| `name` | VARCHAR(100) | - | No | None | User's full name |
| `email` | VARCHAR(255) | UNIQUE | No | None | User's email address |
| `password_hash` | VARCHAR(255) | - | No | None | Bcrypt hashed password |
| `created_at` | DATETIME | - | No | CURRENT_TIMESTAMP | Account creation timestamp |
| `updated_at` | DATETIME | - | No | CURRENT_TIMESTAMP | Profile update timestamp |

### 2. `households` Table

| Column Name | Data Type | Key | Nullable | Default | Description |
| --- | --- | --- | --- | --- | --- |
| `id` | BIGINT UNSIGNED | PK | No | AUTO_INCREMENT | Unique household identifier |
| `user_id` | BIGINT UNSIGNED | FK | No | None | Owner user ID (users.id) |
| `name` | VARCHAR(100) | - | No | None | Household profile name |
| `location` | VARCHAR(120) | - | Yes | NULL | Location / City |
| `total_members` | SMALLINT UNSIGNED | - | No | None | Total household size |
| `earning_members` | SMALLINT UNSIGNED | - | No | 0 | Earning adult count |
| `dependent_members` | SMALLINT UNSIGNED | - | No | 0 | Dependent count |
| `children_count` | SMALLINT UNSIGNED | - | No | 0 | Children count (age < 18) |
| `elderly_count` | SMALLINT UNSIGNED | - | No | 0 | Elderly count (age >= 60) |
| `monthly_income` | DECIMAL(12,2) | - | No | 0.00 | Total monthly income (INR) |

### 3. `expenses` Table

| Column Name | Data Type | Key | Nullable | Default | Description |
| :--- | :--- | :--- | :--- | :--- | :--- |
| `id` | BIGINT UNSIGNED | PK | No | AUTO_INCREMENT | Unique expense ID |
| `household_id` | BIGINT UNSIGNED | FK | No | None | Household ID (households.id) |
| `category` | ENUM(...) | - | No | None | 11 expense categories |
| `amount` | DECIMAL(12,2) | - | No | None | Expense amount |
| `is_essential` | TINYINT(1) | - | No | 1 | 1 = Essential, 0 = Non-essential |
| `description` | VARCHAR(255) | - | Yes | NULL | Expense description |

# CHAPTER 6: TECHNOLOGY STACK AND JUSTIFICATION

## Comprehensive Technology Stack Justification Matrix

| Technology | Purpose | Why Selected | Where Used | How it Works in Project |

| :--- | :--- | :--- | :--- | :--- |

| **React 18** | Frontend UI Framework | Component reusability, virtual DOM performance | frontend/src/ | Renders dynamic single-page UI views |

| **Vite 5** | Frontend Build Tool | Instant server start, fast HMR | frontend/ | Bundles React application for dev & build |

| **Node.js & Express** | Core Backend Runtime | Non-blocking I/O, asynchronous performance | backend/src/ | Handles REST APIs, auth, CRUD, analytics |

| **MySQL 8.0** | Relational Database | Strict ACID compliance, relational foreign keys | database/ | Stores users, households, expenses, predictions |

| **Python FastAPI** | ML Microservice | High-performance asynchronous API, Pydantic | ml-service/ | Exposes /predict endpoint for Random Forest |

| **scikit-learn** | Machine Learning | Robust Random Forest classifier implementation | ml-service/app/ | Trains model, scales 10 features, predicts risk |

| **Google Gemini 2.5** | Conversational AI | Advanced reasoning, natural financial responses | backend/src/services | Generates Q&A answers via floating widget |

| **Spring Boot 3.2** | Read-Only Summary API | Enterprise Java performance, Spring Data JPA | spring-service/ | Exposes /api/spring read-only summary APIs |

| **Chart.js** | Visual Analytics | Responsive 2D HTML5 canvas charts | frontend/src/pages | Displays expenditure distribution & charts |

# CHAPTER 7: DETAILED IMPLEMENTATION

## 7.1 Deterministic Analytics Engine Formulas

### 1. Total Monthly Expenses (E_total)

E_total = SUM(amount_i)

### 2. Essential Expenditure (E_essential) & Non-Essential (E_non_essential)

E_essential = SUM(amount_i where is_essential = 1)

E_non_essential = E_total - E_essential

### 3. Monthly Net Savings (S)

S = I_monthly - E_total

### 4. Savings Rate (R_savings)

R_savings = ((I_monthly - E_total) / I_monthly) * 100 (if I_monthly > 0, else Not Computable)

### 5. Household Dependency Ratio (R_dep)

R_dep = (Dependent_Members / Earning_Members) * 100 (if Earning_Members > 0, else Not Computable)

## 7.2 Machine Learning Microservice Implementation

The Python FastAPI microservice (`ml-service`) loads a trained **Random Forest Classifier** (`rf_model.joblib`) to predict household financial vulnerability.

### 10 Input Features (Strict Order)

1. `total_members`
2. `earning_members`
3. `dependent_members`
4. `children_count`
5. `elderly_count`
6. `monthly_income`
7. `total_expenses`
8. `essential_ratio` (E_essential / E_total)
9. `savings_rate` ((I - E) / I)
10. `expense_income_ratio` (E_total / I)

### Model Performance & Training Specs

- **Training Set:** 1,000 synthetic household profiles (random seed 42).
- **Train/Test Split:** 80% training (800 profiles), 20% testing (200 profiles).
- **Algorithm:** RandomForestClassifier(n_estimators=100, max_depth=6, random_state=42).
- **Accuracy:** **95.0% accuracy** on synthetic test set.
- **Security:** Requires `X-API-Key` header validation.

## 7.3 Floating Gemini AI Chatbot Widget Implementation

Following UI modification requirements, Gemini AI was moved from the sidebar to a **floating ChatGPT-style widget** at the bottom-right corner.

### Component Design (`GeminiChatWidget.jsx`)

- **Positioning:** Fixed circular toggle button (`right: 24px; bottom: 24px; z-index: 1000`) styled with Biscoff dark-peach gradient, bot icon, and hover tooltip "Ask Gemini".
- **Chat Panel:** Opens compact window (`390px` width x `540px` height) with smooth `animate-slide-up` transition.
- **Header:** Features "Gemini AI Assistant", status dot, clear chat (`Trash2`), and close (`X`) buttons.
- **Session Persistence:** Chat history and widget visibility state are stored in `sessionStorage` (`gemini_chat_messages_v1`), preserving active conversations across page navigation.
- **Error Handling & Retry:** Displays readable error alerts with a **Retry** button if API calls fail.

# CHAPTER 8: ANALYTICS AND PREDICTION WORKFLOW

```
[User Enters Expense]
          |
          v
(Node.js Expense API) ---> Save to MySQL Database
          |
          v
(Analytics Engine) ------> Recalculate Savings Rate, Essential %, Dependency Ratio
          |
          v
(Format 10 Features) ---> HTTP POST http://127.0.0.1:8000/predict (X-API-Key)
          |
          v
(FastAPI ML Microservice) -> StandardScale -> RandomForest -> Probability & Class
          |
          v
(Node.js API Response) ---> Save Prediction to MySQL -> Return JSON to React UI
          |
          v
[React Dashboard UI] ----> Render Cards, Charts, Vulnerability Risk Badge
          |
          v
[Platform Floating Chatbot] -> User asks question -> Backend attaches Context -> Gemini Reply
```

# CHAPTER 9: SECURITY, PRIVACY, AND RELIABILITY

1. **Password Protection:** User passwords hashed using `bcryptjs` with 10 salt rounds.
2. **Session Security:** Stateless JWT authentication tokens with expiration limits.
3. **Data Isolation:** Database queries enforce user scoping (`WHERE user_id = ?`).
4. **Secret Protection:** `GEMINI_API_KEY` stored strictly in backend `.env` files; never exposed client-side.
5. **Microservice Authentication:** Python ML service requires `X-API-Key` header validation.
6. **Input Sanitization:** Express-validator and parameterized SQL queries prevent SQL injection and XSS.

# CHAPTER 10: TESTING AND VALIDATION

## 10.1 Automated Test Suite Audit (139 Total Passing Tests)

| Test Suite | Framework / Tool | Test File / Path | Tests Executed | Tests Passed | Status |

| :--- | :--- | :--- | :--- | :--- | :--- |

| **Backend Unit & API Tests** | Jest | backend/tests/*.test.js (7 files) | 109 | 109 | **PASS (100%)** |

| **ML Microservice Tests** | Pytest | ml-service/tests/test_ml_service.py | 21 | 21 | **PASS (100%)** |

| **Frontend Integration Tests**| Vitest | frontend/src/test/frontend.test.jsx | 9 | 9 | **PASS (100%)** |

| **TOTAL** | - | - | **139** | **139** | **PASS (100%)** |

## 10.2 Executed Functional Test Matrix

| Test ID | Module | Feature Tested | Input / Action | Expected Outcome | Status |

| :--- | :--- | :--- | :--- | :--- | :--- |

| **TC-01** | Auth | User Registration | Valid email, name, password | Account created, JWT returned | **PASS** |

| **TC-02** | Auth | Duplicate Email Signup | Existing email | 400 Bad Request, error message | **PASS** |

| **TC-03** | Household | Create Household | Total=4, Earner=2, Dep=2 | Household saved in MySQL | **PASS** |

| **TC-04** | Expense | Add Expense | Category FOOD, INR 5000, Essential | Recalculates expenses & savings | **PASS** |

| **TC-05** | Analytics | Zero Earners Edge Case | Earner=0, Income=0 | Renders "Not Computable" safely | **PASS** |

| **TC-06** | ML Predict | Risk Prediction API | 10-feature payload to FastAPI | Returns HIGH/MEDIUM/LOW & probs | **PASS** |

| **TC-07** | Chatbot | Floating Widget Toggle | Click floating bot button | Opens chat panel smoothly | **PASS** |

| **TC-08** | Chatbot | Gemini Chat Response | Send question in widget | Displays Gemini answer bubble | **PASS** |

| **TC-09** | Chatbot | Sidebar Absence Check | Inspect Sidebar links | Gemini item absent from sidebar | **PASS** |

| **TC-10** | Spring Boot | Financial Summary API | GET /api/spring/households/1/summary | Returns read-only summary JSON | **PASS** |

# CHAPTER 11: RESULTS AND DISCUSSION

The system was fully built, integrated, and validated.

## Objective-to-Result Mapping Matrix

| Project Objective | Audited System Implementation | Result Status |

| :--- | :--- | :--- |

| **Centralized Household Management** | Multi-tenant React dashboard backed by MySQL households table | **ACHIEVED** |

| **Categorized Expense Logging** | 11 categories with essential/non-essential flags in expenses table | **ACHIEVED** |

| **Deterministic Analytics Engine** | Server-side mathematical calculation of savings rate and dependency ratio | **ACHIEVED** |

| **Supervised ML Risk Prediction** | Python FastAPI Random Forest microservice (95.0% synthetic accuracy) | **ACHIEVED** |

| **Floating Gemini AI Chatbot** | Bottom-right floating chatbot widget with sessionStorage persistence | **ACHIEVED** |

| **PDF Report Generation** | Server-side vector PDF generation compiling metrics and recommendations | **ACHIEVED** |

| **Read-Only Spring Boot Service** | Java 17 Spring Boot summary microservice running on port 8081 | **ACHIEVED** |

| **Automated Testing Suite** | 139 passing automated tests (109 Jest, 21 Pytest, 9 Vitest) | **ACHIEVED** |

# CHAPTER 12: DEPLOYMENT AND EXECUTION GUIDE

## Step-by-Step Windows Execution Setup

### 1. Database Initialization (MySQL)

```
mysql -u root -p < "f:\FSD MINI PROJECT\files\database\schema.sql"
```

### 2. Python ML Microservice Setup

```
cd "f:\FSD MINI PROJECT\files\ml-service"
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
python -m uvicorn app.main:app --port 8000 --reload
```

*Service URL:* `http://127.0.0.1:8000` (Health Check: `http://127.0.0.1:8000/health`)

### 3. Node.js Core Backend Setup

```
cd "f:\FSD MINI PROJECT\files\backend"
npm install
npm run dev
```

*Service URL:* `http://localhost:5000`

### 4. Spring Boot Summary Service Setup (Optional Read-Only Service)

```
cd "f:\FSD MINI PROJECT\files\spring-service"
mvn spring-boot:run
```

*Service URL:* `http://localhost:8081/api/spring/health`

### 5. React Frontend Setup

```
cd "f:\FSD MINI PROJECT\files\frontend"
npm install
npm run dev
```

*Application URL:* `http://localhost:5173`

# CHAPTER 13: LIMITATIONS AND FUTURE ENHANCEMENTS

## 13.1 Current Limitations

1. **Synthetic ML Training Dataset:** Model trained on 1,000 synthetic records.
2. **Manual Expense Entry:** Expenses logged manually by users.
3. **External Gemini API Dependency:** AI features require internet connectivity.

## 13.2 Future Enhancements

- **Bank API Integration:** Connect with Open Banking APIs.
- **Time-Series Forecasting:** Implement ARIMA or LSTM models.
- **Multi-Currency Support:** Localized currency conversion and tax estimation.

# CHAPTER 14: CONCLUSION

The **Household Dependency Analytics & Prediction System** successfully delivers a production-grade 3-tier web platform. By combining a Node.js/Express backend, a MySQL database, a Python FastAPI Random Forest machine learning microservice, a read-only Spring Boot service, and a floating Google Gemini AI chatbot widget, the application delivers accurate analytics, predictive risk classification, and interactive decision support.

With **139 automated passing tests**, strict data security, zero-earner edge-case safety, and a modern Biscoff cream user interface, the system achieves all defined project objectives.

# REFERENCES AND APPENDICES

## References


1. Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5-32.

2. CFPB. (2017). *Financial Well-Being in America*. Consumer Financial Protection Bureau.

3. Gemini Team, Google. (2024). *Gemini 1.5 & 2.5 Technical Report*. Google DeepMind.

4. Modigliani, F. (1986). *Life Cycle, Individual Thrift, and the Wealth of Nations*. American Economic Review.

5. Vaswani, A., et al. (2017). *Attention Is All You Need*. NeurIPS 30.